FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	HERNAEZ, Angelee Louise R. **(Solo)**
Section:	AN43
Professor:	Mrs. Crisola Tan
Date:	November 2, 2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description

- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

CHOSEN CASE STUDY: 3: INTELLIGENT TUTORING SYSTEM

Source Code:

```
# ========================================
# CASE-BASED REASONING (CBR) INTELLIGENT TUTORING SYSTEM (ITS)
# ========================================

# 1. CASE BASE: Five Previous Student Cases
case_base = [
    {
        "problem": {
            "student": "Student A",
            "topic": "Algebra",
            "grade_level": "8",
            "error_type": "Incorrect formula",
            "difficulty": "High"
        },
        "solution": {
            "feedback": "Review the correct algebraic formula.",
            "lesson": "Practice solving basic algebraic equations.",
            "intervention": "Provide step-by-step examples."
        }
    },
    {
        "problem": {
            "student": "Student B",
            "topic": "Algebra",
```

```
            "grade_level": "8",
            "error_type": "Sign error",
            "difficulty": "Medium"
        },
        "solution": {
            "feedback": "Pay attention to positive and negative signs.",
            "lesson": "Practice integer operations.",
            "intervention": "Give exercises involving negative numbers."
        }
    },
    {
        "problem": {
            "student": "Student C",
            "topic": "Geometry",
            "grade_level": "9",
            "error_type": "Incorrect formula",
            "difficulty": "High"
        },
        "solution": {
            "feedback": "Review the formulas used in geometry.",
            "lesson": "Study area and perimeter formulas.",
            "intervention": "Provide visual diagrams and examples."
        }
    },
    {
        "problem": {
            "student": "Student D",
            "topic": "Fractions",
            "grade_level": "7",
            "error_type": "Calculation error",
            "difficulty": "Medium"
        },
        "solution": {
            "feedback": "Check each calculation carefully.",
            "lesson": "Practice addition and subtraction of fractions.",
            "intervention": "Provide guided calculation exercises."
        }
    },
    {
        "problem": {
            "student": "Student E",
            "topic": "Algebra",
            "grade_level": "8",
```

```
            "error_type": "Incorrect formula",
            "difficulty": "Medium"
        },
        "solution": {
            "feedback": "Identify the correct formula before solving.",
            "lesson": "Practice substituting values into equations.",
            "intervention": "Provide guided algebra exercises."
        }
    }
]


# 2. SIMILARITY ASSESSMENT
# Each attribute has different importance
weights = {
    "topic": 0.30,
    "grade_level": 0.20,
    "error_type": 0.35,
    "difficulty": 0.15
}


def calculate_similarity(new_problem, old_problem):
    score = 0

    for attribute, weight in weights.items():
        if new_problem[attribute].lower() == old_problem[attribute].lower():
            score += weight

    return score * 100


# 3. RETRIEVE THE MOST SIMILAR CASE
def retrieve_case(new_problem):
    results = []

    for case in case_base:
        old_problem = case["problem"]

        similarity = calculate_similarity(
            new_problem, old_problem
        )
```

```
        results.append({
            "case": case,
            "similarity": similarity
        })

    results.sort(
        key=lambda item: item["similarity"],
        reverse=True
    )

    return results


# 4. ADAPT THE RETRIEVED SOLUTION
def adapt_solution(new_problem, retrieved_case):

    old_solution = retrieved_case["solution"]

    adapted = old_solution.copy()

    if new_problem["error_type"] == "Incorrect formula":
        adapted["feedback"] = (
            "Review the correct formula for "
            + new_problem["topic"]
            + ". Identify the formula before solving."
        )

        adapted["lesson"] = (
            "Study the important formulas in "
            + new_problem["topic"]
            + " and practice applying them."
        )

    elif new_problem["error_type"] == "Sign error":
        adapted["feedback"] = (
            "Pay close attention to positive and negative signs."
        )

        adapted["lesson"] = (
            "Practice exercises involving positive and negative numbers."
        )

    elif new_problem["error_type"] == "Calculation error":
```

```
        adapted["feedback"] = (
            "Review your calculations and solve each step carefully."
        )

        adapted["lesson"] = (
            "Practice step-by-step calculations in "
            + new_problem["topic"] + "."
        )

    if new_problem["difficulty"] == "High":
        adapted["intervention"] = (
            "Provide detailed explanations, guided examples, "
            "and additional practice exercises."
        )

    elif new_problem["difficulty"] == "Medium":
        adapted["intervention"] = (
            "Provide guided exercises followed by independent practice."
        )

    else:
        adapted["intervention"] = (
            "Provide short review exercises and independent practice."
        )

    return adapted


# 5. RETAIN: SAVE THE NEW CASE
def retain_case(new_problem, final_solution):

    new_case = {
        "problem": new_problem.copy(),
        "solution": final_solution.copy()
    }

    case_base.append(new_case)

    print("\nNEW CASE SAVED SUCCESSFULLY!")


# 6. DISPLAY THE UPDATED CASE BASE
def display_case_base():
```

```
    print("\n========== UPDATED CASE BASE ==========")

    for index, case in enumerate(case_base, start=1):

        problem = case["problem"]

        print(f"\nCase {index}")
        print("Student:", problem["student"])
        print("Topic:", problem["topic"])
        print("Grade Level:", problem["grade_level"])
        print("Error Type:", problem["error_type"])
        print("Difficulty:", problem["difficulty"])


# 7. MAIN PROGRAM
def main():

    print("======================================")
    print(" INTELLIGENT TUTORING SYSTEM (ITS)")
    print(" CASE-BASED REASONING PROGRAM")
    print("======================================")

    # Get new student information
    print("\nENTER NEW STUDENT INFORMATION")

    new_problem = {
        "student": input("Student Name: "),
        "topic": input("Learning Topic: "),
        "grade_level": input("Grade Level: "),
        "error_type": input(
            "Error Type (Incorrect formula / Sign error / Calculation error): "
        ),
        "difficulty": input(
            "Difficulty (Low / Medium / High): "
        )
    }

    # Display new problem
    print("\n========== NEW PROBLEM ==========")

    for key, value in new_problem.items():
        print(key.replace("_", " ").title() + ":", value)
```

```
# Retrieve similar cases
results = retrieve_case(new_problem)

print("\n========== SIMILARITY RESULTS ==========")

for index, result in enumerate(results, start=1):

    print(
        f"Rank {index}: "
        f"{result['case']['problem']['student']} - "
        f"{result['similarity']:.2f}% similarity"
    )

# Select the most relevant case
best_match = results[0]["case"]

print("\n========== RETRIEVED CASE ==========")

print("Student:", best_match["problem"]["student"])
print("Topic:", best_match["problem"]["topic"])
print("Error:", best_match["problem"]["error_type"])

# Adapt the retrieved solution
final_solution = adapt_solution(
    new_problem,
    best_match
)

print("\n========== ADAPTED FINAL SOLUTION ==========")

print("Personalized Feedback:",
      final_solution["feedback"])

print("Recommended Lesson:",
      final_solution["lesson"])

print("Teaching Intervention:",
      final_solution["intervention"])

# Retain the new case
retain_case(new_problem, final_solution)
```

```
    # Display updated case base
    display_case_base()

    print("\n====================================")
    print(" CBR PROCESS COMPLETED SUCCESSFULLY")
    print("====================================")


if __name__ == "__main__":
    main()
```

Outputs:

- **New problem description**

```
====================================
 INTELLIGENT TUTORING SYSTEM (ITS)
 CASE-BASED REASONING PROGRAM
====================================

ENTER NEW STUDENT INFORMATION
Student Name: Patrick
Learning Topic: Geometry
Grade Level: 10
Error Type (Incorrect formula / Sign error / Calculation error): Incorrect formula
Difficulty (Low / Medium / High): High

========== NEW PROBLEM ==========
Student: Patrick
Topic: Geometry
Grade Level: 10
Error Type: Incorrect formula
Difficulty: High
```

- **Retrieved similar case**

```
========== SIMILARITY RESULTS ==========
Rank 1: Student C - 80.00% similarity
Rank 2: Student A - 50.00% similarity
Rank 3: Student E - 35.00% similarity
Rank 4: Student B - 0.00% similarity
Rank 5: Student D - 0.00% similarity

========== RETRIEVED CASE ==========
Student: Student C
Topic: Geometry
Error: Incorrect formula
```

- **Adapted final solution**

```
========== ADAPTED FINAL SOLUTION ==========
Personalized Feedback: Review the correct formula for Geometry. Identify the formula before solving.
Recommended Lesson: Study the important formulas in Geometry and practice applying them.
Teaching Intervention: Provide detailed explanations, guided examples, and additional practice exercises.
```

- **Updated case base**

```
NEW CASE SAVED SUCCESSFULLY!

========== UPDATED CASE BASE ==========

Case 1
Student: Student A
Topic: Algebra
Grade Level: 8
Error Type: Incorrect formula
Difficulty: High

Case 2
Student: Student B
Topic: Algebra
Grade Level: 8
Error Type: Sign error
Difficulty: Medium

Case 3
Student: Student C
Topic: Geometry
Grade Level: 9
Error Type: Incorrect formula
Difficulty: High

Case 4
Student: Student D
Topic: Fractions
Grade Level: 7
Error Type: Calculation error
Difficulty: Medium

Case 5
Student: Student E
Topic: Algebra
Grade Level: 8
Error Type: Incorrect formula
Difficulty: Medium

Case 6
Student: Patrick
Topic: Geometry
Grade Level: 10
Error Type: Incorrect formula
Difficulty: High

======================================
 CBR PROCESS COMPLETED SUCCESSFULLY
======================================
```